

HS-Guard Technical Report

Policy-Specific Streaming Moderation with a Hybrid-State Classifier

Zhihao Liu

Naive N0.5

September 2026

Abstract

HS-Guard is a 754.5-million-parameter safety classifier that evaluates prompts and incremental responses without generating text. Its Qwen3.5-based backbone combines eighteen gated-delta layers with six attention layers restricted to a 512-token window. A recurrent-state pool and bounded key/value rings support continuous batching with memory that does not grow with stream length. The released model separates a project-specific red-line decision head from a general safety head. At fixed operating points, the response red-line head detects 112 of 114 positive examples, compared with 105 for Qwen3Guard-Stream-0.6B, while reducing false positives on the corresponding safe-response subset from 45 of 203 to 30. On one NVIDIA L20, the streaming engine has approximately 2.38 ms single-token median service latency and supports 352 simulated concurrent streams with worst-seed P95 latency below 18.37 ms across initial contexts of 512, 2,048 and 8,000 tokens. We release the inference engine, model weights, evaluation summaries and this report. The measurements characterize specified policies and implementations rather than universal safety or deployment performance.

Code: <https://github.com/KrisLiu16/HS-Guard>

Weights: <https://huggingface.co/KrisLiu16/HS-Guard-v10>

1 Introduction

A streaming application receives model output before the response is complete. Moderation therefore has two separate requirements: deciding which content warrants intervention and returning that decision quickly enough to act on a live stream. A policy-specific intervention rule also differs from a general harmfulness benchmark. Treating the two as one label space makes both evaluation and calibration difficult to interpret.

HS-Guard addresses these requirements with a hybrid-state backbone and separate classification heads. The release includes a low-level stable-token streaming interface and a complete-message reference scorer. The streaming engine uses fixed slot-indexed state, bucketed batches, CUDA Graph replay and fused Triton kernels. It performs classification only; there is no vocabulary projection or autoregressive token generation.

This report describes the final v10 checkpoint and its measured inference engine. It replaces development-stage results with a fixed release identity. Earlier experiments informed the architecture and training recipe, but their operating points are not mixed with the final results.

2 Model and Decision Interface

2.1 Hybrid-state backbone

The backbone is derived from Qwen3.5-0.8B-Base [2]. All 24 text layers are retained: 18 gated-delta layers and 6 gated attention layers. Direct attention is restricted to a sliding window of $W = 512$ tokens.

Earlier context is represented through the gated-delta recurrence, not an unbounded attention cache. The released checkpoint contains 754,519,918 parameters, including both classification branches.

A simplified gated-delta update is

$$\tilde{S}_{t-1} = e^{g_t} S_{t-1}, \quad (1)$$

$$S_t = \tilde{S}_{t-1} + \beta_t k_t \left(v_t - \tilde{S}_{t-1}^\top k_t \right)^\top, \quad (2)$$

$$o_t = S_t^\top q_t. \quad (3)$$

The decay, update gate and projected vectors are input dependent. Each live session stores recurrent matrices, short-convolution history and bounded attention rings. Increasing the number of processed tokens changes the state values and position indices, but not the allocated state shape.

2.2 Two roles and two policies

Prompt and response roles have separate readouts. For each role, a 512-dimensional projection, layer normalization and SiLU activation feed a three-class risk layer. The classes are ordered as safe, unsafe and controversial. A separate general-safety projection and readout share the same backbone. Category tensors remain in the checkpoint for compatibility; category accuracy is not a validated release feature.

The intervention score is

$$s_{\text{cut}}(t) = 1 - p(\text{safe} \mid x_{\leq t}). \quad (4)$$

For a complete prompt, the decision uses the final content-token score. For a response, the evaluated rule uses the maximum score over its content tokens. Both unsafe and controversial probability contribute to intervention. Thresholds are branch- and role-specific.

The red-line branch implements a project-specific policy covering designated violence, weapon and drug facilitation, explicit sexual content, self-harm methods, and selected political or public-order categories. Its treatment of quoted or restated content can differ from general moderation labels. Harm outside the policy can receive an intermediate alert target. The general branch is intended for broader benchmark-compatible harmfulness scoring. These branches are evaluated separately.

2.3 Serialization and serving contract

Messages are serialized as the upper-case role, a colon and newline, followed by content; messages are separated by a blank line. The released tokenizer must be used without an alternative chat template. Complete-message scoring identifies the last message’s content positions and excludes earlier turns from the final risk aggregation.

The streaming interface accepts stable token IDs. Upstream text chunk boundaries are not necessarily tokenizer boundaries: an integration must delay unstable suffix tokens or restore and replay a saved state after retokenization. The low-level engine does not automatically solve arbitrary text rollback. A session slot belongs to one conversation until its state is reset.

3 Training and Calibration

3.1 Backbone adaptation and general safety

The shared backbone and general branch are inherited from the preceding full adaptation stage. That stage used 170,512 training records, approximately 21.6 million input tokens per pass, and two epochs. General-safety supervision included distillation from Qwen3Guard-Stream-4B. Public benchmark cases were partitioned by the first user message before that distillation; evaluation below uses the held partition. The final prompt-focused stages do not modify the backbone or general branch.

3.2 Policy labels

A structured judge records content attributes such as topic, act, detail and variant spelling. A policy table maps these attributes to severity and reporting strata. For earlier token-supervised stages, clause-level prefix probes locate transitions and exclude positions whose labels are not determined. Non-monotonic probe sequences and failed judgments are excluded. This separates policy interpretation from the underlying recorded attributes.

The final prompt update uses end-of-prompt binary supervision. Unsafe and controversial targets are combined into the positive class. Ordinary safe examples have target zero; alert examples use an intermediate cut target of 0.3. Automated labels and agent spot checks are not an independent human gold standard.

3.3 Final prompt update

A pool of 24,798 public training prompts was filtered against prior training, calibration, development and benchmark prompt families. The checks use normalized text, punctuation-free word sequences and a shared 16-word prefix; they do not establish complete semantic or pretraining decontamination. A deterministic source/length-balanced selection produced 6,000 prompts, split before labeling into 4,514 training, 738 calibration and 748 audit examples.

Of the 6,000 labels, 5,900 were usable. An agent reviewed 60 training examples and excluded eight ambiguous records. The final mixture contains 67,341 training records, including 4,430 new prompts. Calibration contains 4,186 records, including 728 new examples. The audit contains 734 usable examples; 14 of its preselected examples have unusable labels.

Only the prompt projection and risk layer are updated: 527,363 trainable parameters. Training runs for two epochs with AdamW, learning rate 5×10^{-5} , effective batch size 16 and length-bucketed microbatches. New prompts contribute one third of the source-level loss mixture. The run processes 8,148,474 tokens in 8,418 updates. Frozen-tensor hashes before and after training agree.

3.4 Selection protocol

Five checkpoints, including initialization, are calibrated. For each checkpoint, the prompt threshold must satisfy an empirical 15% normal-example false-positive budget independently on the legacy and new calibration domains. The selected model maximizes the equally weighted mean of the two red-line recall values. The final update is selected. Audit labels and benchmark labels do not select the checkpoint or threshold.

The released red-line thresholds are 0.0761368871 for prompts and 0.1117895246 for responses. The response threshold and response tensors are unchanged from the earlier response-focused model. The general branch uses 0.5. A calibration budget is a selection rule, not a guarantee on a shifted test distribution.

4 Evaluation

4.1 Benchmark construction

The public English suite follows the tasks in the Qwen3Guard technical report [1]. It contains 18,098 prompt and response cases across 13 benchmark tasks. The held partition contains 9,461 cases; Think is held in full. WildGuard tasks use an English PolyGuard proxy because the original source was gated. XSTest responses are not reproduced. Original paper scores, proxy scores and held-partition scores are therefore not interchangeable.

Qwen3Guard-Stream-0.6B is run on the same cases at its original 0.5 threshold. Prompt scores use the final token; response scores use the maximum token score. Public benchmark F1 uses the original harmfulness labels. The project-policy subsets instead use policy-specific labels. The previously observed held suite is useful for version comparison, but does not constitute a new independent confirmation set.

4.2 Response-focused operating point

Table 1 presents the response operating point emphasized by this release. The final model detects seven additional red-line responses while producing fifteen fewer false positives on the specified safe-response subset. Counts and denominators are retained because percentage changes alone can obscure the size of the test.

Table 1: Response classification under the project policy. Recall is higher-is-better; false-positive rate is lower-is-better. Fixed thresholds: HS-Guard 0.1117895, Qwen3Guard 0.5.

Metric	Qwen3Guard-Stream-0.6B	HS-Guard v10
Red-line recall ($n = 114$)	105/114 (92.11%)	112/114 (98.25%)
Safe-response false positives ($n = 203$)	45/203 (22.17%)	30/203 (14.78%)
Missed red-line responses	9	2

The observed recall improvement is 6.14 percentage points. The false-positive count is reduced by 33.3% relative to this baseline count, and missed positives by 77.8%. These are measurements on the named subsets, not estimates of the error rate on all real-world traffic. A paired, example-level bootstrap with 5,000 resamples gives an exploratory recall-difference interval of [1.75,11.40] percentage points and a false-positive-rate difference interval of [-13.79,-1.48]. These intervals do not adjust for repeated experiment selection or multiple comparisons.

All 9,204 response benchmark scores match the preceding response checkpoint exactly. The normal probe set produces two flags out of 194 examples on each role. A separate targeted probe detects all 64 designated positive examples and flags four of 67 normal controls.

4.3 General branch and audit

The backbone and general branch are frozen, so their previously verified held scores remain applicable to the release. Two selected improvements are ToxicChat prompt F1, 74.16 versus 70.51 ($n = 1,373$), and Aegis 2.0 response F1, 81.65 versus 78.95 ($n = 405$). Full per-benchmark tables and policy-subset results accompany the release in machine-readable form. Scores from the general branch are not substituted for red-line branch scores.

The fresh audit is evaluated only after selection. It contains 87 positive, 389 ordinary-safe and 258 other policy-pass prompts. HS-Guard detects 84 of 87 positives. This equals the recalibrated initialization’s positive count; the audit does not establish that the final update improves recall on new data. The labels are automated, and policy-pass false positives remain substantial. The released evaluation summaries include all three strata and the same-example Qwen baseline.

5 Streaming Inference Engine

5.1 State layout and continuous batching

The engine allocates state pools indexed by session slot. Gated-delta matrices use FP16 storage with FP32 kernel accumulation. Convolution history and attention key/value rings use BF16. The attention

window has a scratch column for padded writes, and padded sessions use a dedicated scratch slot. Padding does not commit a state update.

Each scheduler tick groups pending tokens into fixed session and token buckets. A CUDA Graph captures the tensor operations for each shape. Inputs are packed into pinned host buffers and copied to the GPU; probability outputs are copied back before timing stops. Appends longer than the largest token bucket are split into smaller ticks. The API exposes classification probabilities rather than generated text.

The fused implementation combines convolution and activation, gated-delta state updates, normalization glue, rotary transforms and in-place ring attention. It avoids repeated state gathers and transient tensor copies. The gated-delta kernels contain adaptations of the MIT-licensed flash-linear-attention implementation [3]; notices are included in the source distribution.

5.2 Measurement protocol

Performance measurements use one NVIDIA L20, PyTorch 2.7.1 with CUDA 12.8, the released v10 tensors and the fused engine. Service latency measures back-to-back single-token appends after prefill, with 20 warmup and 200 measured appends. Arrival tests use per-session rates drawn uniformly from 30–60 tokens/s, independent offsets up to one second, ten seconds per measurement and two seeds. The benchmark uses repeated tokenized conversations to provide sufficiently long streams.

Latency is measured from token arrival to the verdict being available on the host. Prefill, tokenizer work, transport and cold start are excluded. Context lengths count each model’s own tokenizer tokens. The Qwen comparison reuses a same-L20 measurement of its support branch at commit 9a06537, SGLang 0.5.3rc0, with radix caching enabled. It is a comparison of those implementations, not an upper bound on all Qwen serving systems.

Table 2: Single-stream median service latency, milliseconds per appended token. The speed ratio is baseline latency divided by HS-Guard latency.

Initial tokens	Qwen SGLang + cache	HS-Guard v10	Ratio
512	11.652	2.384	4.89×
2,048	11.671	2.383	4.90×
8,000	12.658	2.383	5.31×

At all three context lengths, 352 tested sessions meet the 20 ms P95 target. Across their six seed/context measurements, worst P95 is 18.368 ms and minimum measured throughput is 15,075.6 tokens/s. The next tested point, 384 sessions, exceeds the target. These are short synthetic-arrival tests, not a production HTTP concurrency promise.

5.3 Numerical verification

On 64 fixed, evenly spaced benchmark cases, the final flag decisions of the release-stage engine agree with the full-forward reference. The maximum absolute per-token cut-score difference is 0.020701. This supports the tested operating point but is not bitwise equivalence of every output or a full-suite accuracy proof. An isolated-package check additionally reproduces the original single-record reference exactly on 64 cases for both classification branches. Against the archived batched scores, the largest differences are 0.00645 and 0.00503, with no changed decisions. A 16-case streaming package check also preserves all decisions. This matched-control test distinguishes packaging errors from batch-dependent floating-point variation.

6 Release Scope and Limitations

The source release contains the classifier loader, reference window attention, streaming kernels, example usage and evaluation summaries. The model repository contains complete weights, tokenizer assets, fixed thresholds and checksums. Deployment credentials, internal infrastructure, raw training and evaluation text, and private policy dictionaries are excluded.

Code is distributed under Apache-2.0, with third-party MIT notices. The weight release is restricted to noncommercial research use because the training provenance includes noncommercially licensed sources such as ToxicChat [4]. No training dataset is redistributed. Upstream base-model attribution and licensing notices remain applicable.

Coverage is limited to the described English benchmark reconstruction, project-policy subsets and targeted probes. The release does not establish complete multilingual, category-label or refusal-label accuracy. Fixed-size state controls memory growth, but it does not preserve all historical information as full attention would. Longer-running load tests, production tokenization integration and independently annotated policy evaluations remain separate validation tasks.

7 Conclusion

HS-Guard combines a policy-specific classifier with a bounded-state streaming runtime. The release demonstrates a favorable response-recall/false-positive operating point on the specified project-policy subsets and low-millisecond incremental inference in the measured L20 implementation. The published model, source and measurement summaries make these claims inspectable without distributing internal deployment material.

References

- [1] Qwen Team. *Qwen3Guard Technical Report*. arXiv:2510.14276, 2025. <https://arxiv.org/abs/2510.14276>.
- [2] Qwen Team. *Qwen3.5-0.8B-Base model card and license*. <https://huggingface.co/Qwen/Qwen3.5-0.8B-Base>.
- [3] Songlin Yang, Yu Zhang, Zhiyuan Li and contributors. *flash-linear-attention*. <https://github.com/fla-org/flash-linear-attention>.
- [4] Zi Lin et al. *ToxicChat: Unveiling Hidden Challenges of Toxicity Detection in Real-World User-AI Conversation*. 2023. Dataset: <https://huggingface.co/datasets/lmsys/toxic-chat>.